

1. Transition — सभी important properties और methods

[Share](#) ...

CSS Property / Method	क्या काम करता है	Example	Deep Explanation
<code>transition</code>	Transition की सारी settings का shorthand	<code>transition: all 0.5s ease;</code>	एक ही declaration में property + duration + timing-function + delay सेट कर सकते हो।
<code>transition-property</code>	कौन-सी CSS property smoothly change होगी	<code>transition-property: transform;</code>	<code>transform</code> , <code>opacity</code> , <code>background-color</code> , <code>width</code> जैसी properties चुन सकते हो। <code>all</code> देने पर सभी applicable properties पर transition लगाया जा सकता है।
<code>transition-duration</code>	Change होने में कितना समय लगेगा	<code>transition-duration: 2s;</code>	<code>2s</code> मतलब transition को पूरा होने में 2 seconds लगेगे। <code>500ms</code> = 0.5 second।
<code>transition-delay</code>	Transition शुरू होने से पहले कितना wait	<code>transition-delay: 1s;</code>	Hover करते ही effect शुरू होने के बजाय 1 second बाद शुरू होगा।
<code>transition-timing-function</code>	Transition की speed कैसे बदलेगी	<code>transition-timing-function: ease;</code>	शुरुआत/अंत में speed slow या fast करने का तरीका तय करता है।
<code>transition-behavior</code>	Discrete properties के transition behavior को control	<code>transition-behavior: allow-discrete;</code>	कुछ ऐसी properties जो normally smooth होतीं, उनके transition behavior को allow कर सकता है।
<code>steps()</code>	Transition को fixed steps में चलाता है	<code>transition-timing-function: steps(5);</code>	Smooth movement की जगह 5 अलग-अलग jumps/stages मिलते हैं।
<code>cubic-bezier()</code>	Custom speed curve बनाता है	<code>cubic-bezier(.2,.8,.2,1)</code>	<code>ease</code> से ज्यादा precise acceleration/deceleration control मिलता है।

Transition Timing Methods

Value	क्या करता है
<code>linear</code>	पूरी transition में constant speed
<code>ease</code>	शुरुआत slow → बीच में fast → अंत में slow
<code>ease-in</code>	शुरुआत slow → बाद में fast
<code>ease-out</code>	शुरुआत fast → अंत में slow
<code>ease-in-out</code>	शुरुआत slow → middle fast → end slow
<code>step-start</code>	शुरुआत में तुरंत final state
<code>step-end</code>	transition के अंत में final state
<code>steps(3)</code>	3 fixed stages
<code>steps(10)</code>	10 fixed stages

Multiple Transition

Syntax	Meaning	📄
<code>transition: width 1s ease;</code>	केवल width	
<code>transition: width 1s, height 2s;</code>	width और height अलग-अलग	
<code>transition: transform .5s, opacity 1s;</code>	transform और opacity की अलग duration	
<code>transition: all .5s ease-in-out;</code>	सभी applicable properties	

2. Transform — सभी 2D और 3D methods

[Share](#)

Transform Method	क्या करता है	Example	Deep Explanation
<code>rotate()</code>	Element को rotate करता है	<code>transform: rotate(45deg);</code>	Positive degree सामान्यतः clockwise और negative degree opposite direction में rotate करता है।
<code>rotateX()</code>	X-axis पर rotate	<code>transform: rotateX(45deg);</code>	Horizontal axis के around 3D rotation।
<code>rotateY()</code>	Y-axis पर rotate	<code>transform: rotateY(45deg);</code>	Vertical axis के around 3D rotation।
<code>rotateZ()</code>	Z-axis पर rotate	<code>transform: rotateZ(45deg);</code>	Z-axis rotation; 2D rotation जैसा दिखाई देता है।
<code>scale()</code>	X और Y दोनों directions में size बदलता है	<code>transform: scale(2);</code>	2 का मतलब दोनों directions में size 2 गुना।
<code>scale()</code>	X और Y दोनों directions में size बदलता है	<code>transform: scale(2);</code>	2 का मतलब दोनों size 2 गुना।
<code>scale(x,y)</code>	X/Y scaling अलग-अलग	<code>transform: scale(2,.5);</code>	Width 2× और height 0.5× हो सकती है।
<code>scaleX()</code>	केवल horizontal scaling	<code>transform: scaleX(2);</code>	Width direction में scaling।
<code>scaleY()</code>	केवल vertical scaling	<code>transform: scaleY(2);</code>	Height direction में scaling।
<code>scaleZ()</code>	Z-axis scaling	<code>transform: scaleZ(2);</code>	3D depth direction में scaling।
<code>scale3d()</code>	X/Y/Z तीनों में scaling	<code>transform: scale3d(2,1,3);</code>	तीनों axes पर अलग-अलग scaling।
<code>translate()</code>	X और Y पर move	<code>transform: translate(100px,50px);</code>	Element को horizontal और vertical direction में visually move करता है।
<code>translateX()</code>	X-axis पर move	<code>transform: translateX(200px);</code>	Right/left movement।

<code>translateY()</code>	Y-axis पर move	<code>transform:</code> <code>translateY(100px);</code>	Down/up movement
<code>translateZ()</code>	Z-axis पर move	<code>transform:</code> <code>translateZ(100px);</code>	3D depth direction में movement।
<code>translate3d()</code>	X/Y/Z movement	<code>transform:</code> <code>translate3d(100px, 50px, 20px);</code>	तीन dimensions में movement।
<code>skew()</code>	X और Y दोनों पर tilt	<code>transform:</code> <code>skew(30deg, 10deg);</code>	Box को तिरछा/slant करता है।
<code>skewX()</code>	X direction में skew	<code>transform: skewX(30deg);</code>	Horizontal slant।
<code>skewY()</code>	Y direction में skew	<code>transform: skewY(30deg);</code>	Vertical slant।
<code>matrix()</code>	Advanced 2D transformation	<code>transform: matrix(...);</code>	Translate, scale, skew और rotate को mathematical matrix से represent करता है।
<code>matrix3d()</code>	Advanced 3D transformation	<code>transform: matrix3d(...);</code>	3D transformations को 4x4 matrix के रूप में define करता है।
<code>perspective()</code>	Transform के अंदर 3D perspective	<code>transform:</code> <code>perspective(500px)</code> <code>rotateY(45deg);</code>	3D object को depth वाला visual effect देता है।

3. Transform की supporting properties

Property	क्या काम करता है	Example	Deep Explanation
<code>transform</code>	Transform apply करता है	<code>transform: rotate(45deg);</code>	इसी property में <code>rotate</code> , <code>scale</code> , <code>translate</code> , <code>skew</code> आदि functions लगते हैं।
<code>transform-origin</code>	Transform किस point से होगा	<code>transform-origin: center;</code>	Rotation/scale का center बदल देता है। <code>top left</code> , <code>center</code> , <code>bottom right</code> , % आदि दे सकते हो।
<code>transform-style</code>	Child elements का 3D behavior	<code>transform-style: preserve-3d;</code>	Parent के अंदर children के 3D transforms को preserve करने के लिए।
<code>perspective</code>	3D depth define करता है	<code>perspective: 800px;</code>	Parent के children को 3D viewing perspective देता है।
<code>perspective-origin</code>	Perspective का viewing point	<code>perspective-origin: center;</code>	3D scene किस point से देखी जा रही है, यह control करता है।

`backface-visibility`

Element का backside
दिखाना/छिपाना

`backface-visibility:`
`hidden;`

Card flip जैसे 3D effects में
backside को hide करने के लिए।

Transform Origin Values

Value	Meaning
<code>center</code>	Center से transform
<code>top</code>	Top से
<code>bottom</code>	Bottom से
<code>left</code>	Left से
<code>right</code>	Right से
<code>top left</code>	↓ Top-left corner से

top left	Top-left corner से
50% 50%	Exact center
0 0	Top-left origin
100% 100%	Bottom-right origin

4. Transform को combine करना

Code	क्या होगा	
transform: translateX(100px) rotate(45deg);	पहले translation/transform chain के बाद rotation	
transform: scale(2) rotate(45deg);	Size बढ़ेगा और rotate होगा	
transform: translate(50px, 20px) scale(1.5);	Move + scale	

```
transform: scale(2) rotate(45deg);
```

Size बढ़ेगा और rotate होगा

```
transform: translate(50px,20px)
```

Move + scale

```
scale(1.5);
```

```
transform: rotate(45deg) scale(2);
```

Rotate + scale

```
transform: translateX(100px) skewX(20deg);
```

Move + horizontal skew

Important Point

Explanation

Multiple transforms

Space से functions लिख सकते हो।

Order important है

`translate() rotate()` और `rotate() translate()` का result अलग हो सकता है।

`transform` layout को normally reflow नहीं करता

Visual transformation होती है; original layout position/space का behavior सामान्य width/height बदलने जैसा नहीं होता।

5. Box Shadow — सभी important options

Property / Method	क्या करता है	Example	Deep Explanation	
<code>box-shadow</code>	Element के बाहर/ अंदर shadow	<code>box-shadow: 5px 8px 9px red;</code>	Shadow की position, blur, spread और color define कर सकते हो।	
X-offset	Horizontal shadow position	<code>5px</code>	Positive = right, negative = left।	
Y-offset	Vertical shadow position	<code>8px</code>	Positive = नीचे, negative = ऊपर।	
Blur radius	Shadow को soft करता है	<code>10px</code>	ज्यादा blur = ज्यादा soft/faded shadow।	
Spread radius	Shadow का size बढ़ाता/घटाता	<code>3px</code>	Positive value shadow expand करती है; negative value shrink कर सकती है।	
Color	Shadow का रंग		Shadow का color।	

<code>inset</code>	Shadow अंदर बनाता है	<code>inset 0 0 10px gray</code>	Normal shadow बाहर होती है, <code>inset</code> अंदर shadow बनाता है।
Multiple shadows	एक से ज्यादा shadows	<code>0 2px 5px red, 0 5px 10px blue</code>	Comma लगाकर multiple shadows बना सकते हो।
box-shadow Syntax			
Syntax	Meaning 		
<code>box-shadow: 5px 8px;</code>	X + Y offset		
<code>box-shadow: 5px 8px 10px;</code>	X + Y + blur		
<code>box-shadow: 5px 8px 10px red;</code>	X + Y + blur + color		
<code>box-shadow: 5px 8px 10px 3px red;</code>	X + Y + blur + spread + color		
<code>box-shadow: inset 0 0 10px black;</code>	Inner shadow		
<code>box-shadow: none;</code>	Shadow remove 		

6. Background — Complete Properties

[Share](#)

Property	क्या करता है	Example	Deep Explanation
<code>background</code>	Background का shorthand	<code>background: red url(bg.jpg) center/cover no-repeat;</code>	Color, image, position, size, repeat आदि को एक declaration में लिख सकते हो।
<code>background-color</code>	Background color	<code>background-color: yellow;</code>	Element के पीछे base color set करता है।
<code>background-image</code>	Background image	<code>background-image: url("bg.jpg");</code>	Element के background में image लगाता है।
<code>background-size</code>	Background image का size	<code>background-size: cover;</code>	Image container को कैसे fill करेगी, तय करता है।
<code>background-repeat</code>	Background image repeat	<code>background-repeat: no-repeat;</code>	Image repeat होगी या नहीं।
<code>background-position</code>	Image की position	<code>background-position: center;</code>	Background image को container में कहाँ रखा जाएगा।

<code>background-position</code>	Image की position	<code>background-position: center;</code>	Background image को में कहाँ रखा जाएगा।
<code>background-attachment</code>	Background का scrolling behavior	<code>background-attachment: fixed;</code>	Scroll करने पर background fixed/scroll behavior control करता है।
<code>background-origin</code>	Background positioning area	<code>background-origin: border-box;</code>	Background image positioning किस box से calculate होगी।
<code>background-clip</code>	Background painting area	<code>background-clip: padding-box;</code>	Background कहाँ तक paint होगी।
<code>background-blend-mode</code>	Background layers को blend करता है	<code>background-blend-mode: multiply;</code>	Multiple backgrounds/color को blending mode से combine करता है।

[Share](#)

7. background-size — सभी important values

Value	क्या करता है
auto	Image का original/intrinsic size maintain करता है।
cover	पूरा container cover करता है; image का कुछ हिस्सा crop हो सकता है।
contain	पूरी image दिखाता है; container में खाली space बच सकता है।
100% 100%	Image को container की पूरी width और height में stretch करता है।
50% 50%	Image की width/height को percentage से control कर सकते हो।
200px 100px	Exact width और height दे सकते हो।

cover vs contain

cover	contain
Container पूरा भरता है	पूरी image दिखाता है
Image crop हो सकती है	Empty space बच सकता है
Hero/banner में बहुत useful	Product/image preview में useful

8. background-repeat — सभी values

Value	क्या करता है
repeat	X और Y दोनों direction में repeat
repeat-x	केवल horizontal repeat
repeat-y	केवल vertical repeat
no-repeat	बिल्कुल repeat नहीं
space	Tiles के बीच available space distribute
round	Image को adjust करके repeat करता है

9. background-position — सभी common methods

Value	क्या करता है
left	Left
center	Center
right	Right
top	Top
bottom	Bottom
left top	Top-left
right top	Top-right
left bottom	Bottom-left
right bottom	Bottom-right



<code>center center</code>	Exact center
<code>50% 50%</code>	Percentage-based center
<code>20px 30px</code>	Exact X/Y position

10. Multiple Background Images

Method	Example	Deep Explanation
Multiple images	<code>background-image: url(a.png), url(b.png);</code>	एक ही element पर multiple background layers।
Multiple positions	<code>background-position: left top, right bottom;</code>	प्रत्येक image की अलग position।
Multiple sizes	<code>background-size: 100px, cover;</code> 	प्रत्येक background की अलग size।

Multiple repeat	<code>background-repeat: no-repeat, repeat;</code>	हर layer का अलग repeat behavior।	Share
Multiple colors/layers	<code>background: url(a), url(b), red;</code>	Multiple images के नीचे background color भी रख सकते हो।	

11. Gradient Methods

Method	क्या करता है	Example	Deep Explanation	
<code>linear-gradient()</code>	Straight direction में gradient	<code>linear-gradient(red, blue)</code>	एक color से दूसरे color में linear transition।	
<code>radial-gradient()</code>	Center से circular/elliptical gradient	<code>radial-gradient(red, blue)</code>	Center से बाहर की तरफ colors spread होते हैं।	
<code>conic-gradient()</code>	Center के around angular gradient	<code>conic-gradient(red, yellow, blue)</code>	Circular/angular direction में color transition।	

<code>linear-gradient()</code>	Straight direction में gradient	<code>linear-gradient(red, blue)</code>	एक color से दूसरे color में transition।
<code>radial-gradient()</code>	Center से circular/elliptical gradient	<code>radial-gradient(red, blue)</code>	Center से बाहर की तरफ colors spread होते हैं।
<code>conic-gradient()</code>	Center के around angular gradient	<code>conic-gradient(red, yellow, blue)</code>	Circular/angular direction में color transition।
<code>repeating-linear-gradient()</code>	Repeating linear pattern	<code>repeating-linear-gradient(red 0 10px, blue 10px 20px)</code>	Linear gradient repeatedly repeat होता है।
<code>repeating-radial-gradient()</code>	Repeating radial pattern	<code>repeating-radial-gradient(red 0 10px, blue 10px 20px)</code>	Circular pattern repeat होता है।
<code>repeating-conic-gradient()</code>	Repeating conic pattern	<code>repeating-conic-gradient(red 0deg 20deg, blue 20deg 40deg)</code>	Angular pattern repeat होता है।

12. Position — सभी properties



Property	क्या करता है	Example	Deep Explanation
<code>position</code>	Element का positioning mode तय करता है	<code>position: relative;</code>	<code>static</code> , <code>relative</code> , <code>absolute</code> , <code>fixed</code> , <code>sticky</code> मुख्य values हैं।
<code>top</code>	Top offset	<code>top: 50px;</code>	Positioned element को reference point से नीचे/ऊपर offset कर सकता है।
<code>right</code>	Right offset	<code>right: 20px;</code>	Right side से positioning offset।
<code>bottom</code>	Bottom offset	<code>bottom: 20px;</code>	Bottom side से positioning offset।
<code>left</code>	Left offset	<code>left: 50px;</code>	Left side से positioning offset।
<code>inset</code>	चारों offsets का shorthand	<code>inset: 10px;</code>	<code>top/right/bottom/left</code> को एक साथ set करता है।



<code>inset</code>	चारों offsets का shorthand	<code>inset: 10px;</code>	<code>top/right/bottom/left</code> को एक साथ set करता है।
<code>inset-block</code>	Block direction के offsets	<code>inset-block: 20px;</code>	Writing mode के अनुसार block-axis दोनों sides को control करता है।
<code>inset-inline</code>	Inline direction के offsets	<code>inset-inline: 20px;</code>	Writing mode के अनुसार inline-axis दोनों sides को control करता है।
<code>inset-block-start</code>	Block-start offset	<code>inset-block-start: 10px;</code>	Logical top-equivalent direction।
<code>inset-block-end</code>	Block-end offset	<code>inset-block-end: 10px;</code>	Logical bottom-equivalent direction।
<code>inset-inline-start</code>	Inline-start offset	<code>inset-inline-start: 10px;</code>	Logical left/right start direction।
<code>inset-inline-end</code>	Inline-end offset	<code>inset-inline-end:</code>	Logical opposite inline direction।

`z-index`

Overlapping
elements का
stacking order

`z-index: 10;`

Higher stacking level वाला
element ऊपर आ सकता है, लेकिन
stacking contexts भी matter करते
हैं।

13. `position` के सभी मुख्य values

Value	क्या करता है	कब use करते हैं	
<code>static</code>	Normal document flow	Default positioning	
<code>relative</code>	अपनी original जगह रखता है और offset किया जा सकता है	Parent को positioning reference बनाने के लिए भी बहुत common	
<code>absolute</code>	Normal flow से बाहर निकलता है	Badge, icon, overlay, dropdown, child positioning	
<code>fixed</code>	Viewport के relative position	Fixed navbar, floating button, chat	



fixed	Viewport के relative position	Fixed navbar, floating button, chat button	Share
sticky	Scroll के दौरान threshold तक normal, फिर sticky	Sticky navbar/sidebar	

14. relative vs absolute vs fixed vs sticky

Feature	relative	absolute	fixed	sticky
Normal flow	हाँ	नहीं	नहीं	हाँ
Original space रहती है	हाँ	नहीं	नहीं	हाँ
top/left काम करते हैं	हाँ	हाँ	हाँ	हाँ
Common reference	अपनी original position	↓ containing block	viewport/containing context	scroll container + threshold

Common reference	अपनी original position	containing block	viewport/containing context	scroll container + threshold
Scroll पर normally चलता है	हाँ	सामान्य document के साथ	viewport में fixed	threshold तक, फिर sticky
Common use	Positioning parent	Overlay/badge	Navbar/floating button	Sticky header

15. z-index

Concept	Example	Deep Explanation
Basic	<code>z-index: 10;</code>	Overlapping elements में stacking order control करता है।
Negative	<code>z-index: -1;</code>	Element को lower stacking level पर ला सकता है; stacking context/background behavior पर ध्यान देना जरूरी है।
Higher value	<code>z-index: 100;</code>	Same relevant stacking context में lower value वाले item से ऊपर आ सकता है।
<code>auto</code>	<code>z-index: auto;</code>	Default stacking behavior।
Stacking context	<code>.box { position: relative; z-index: 5; }</code>	<code>z-index</code> केवल simple global number comparison नहीं है; stacking contexts के कारण nested elements अलग context में रह सकते हैं।

16. Flexbox — सभी Container Properties

Property	क्या करता है	Example	Deep Explanation	
<code>display</code>	Flex container बनाता है	<code>display: flex;</code>	इसके children flex items बन जाते हैं।	
<code>flex-direction</code>	Main axis की direction	<code>flex-direction: row;</code>	Items row या column में arrange होते हैं।	
<code>flex-wrap</code>	Items multiple lines में जाएँ या नहीं	<code>flex-wrap: wrap;</code>	Space कम होने पर items next line में जा सकते हैं।	
<code>flex-flow</code>	Direction + wrap shorthand	<code>flex-flow: row wrap;</code>	<code>flex-direction</code> + <code>flex-wrap</code> का shorthand।	
<code>justify-content</code>	Main axis पर distribution	<code>justify-content: center;</code>	Items को main axis पर start/end/center/space के अनुसार distribute करता है।	
<code>align-items</code>	Cross axis पर items align	 <code>align-items: center;</code>	प्रत्येक flex line के अंदर items की cross-axis alignment।	

<code>justify-content</code>	Main axis पर distribution	<code>justify-content: center;</code>	Items को main axis पर start/end/center/space के अनुसार distribute करता है।
<code>align-items</code>	Cross axis पर items align	<code>align-items: center;</code>	प्रत्येक flex line के अंदर items की cross-axis alignment।
<code>align-content</code>	Multiple flex lines को distribute	<code>align-content: space-between;</code>	Multiple lines होने पर उनके बीच/around extra space distribute करता है।
<code>gap</code>	Rows/columns के बीच gap	<code>gap: 20px;</code>	Flex items के बीच consistent spacing।
<code>row-gap</code>	Row/line gap	<code>row-gap: 20px;</code>	Flex lines के बीच gap।
<code>column-gap</code>	Column/item gap	<code>column-gap: 20px;</code>	Horizontal gap।

17. `flex-direction` — सभी values

Value	क्या करता है
<code>row</code>	Main axis horizontal; सामान्यतः left → right
<code>row-reverse</code>	Horizontal direction reverse
<code>column</code>	Main axis vertical; top → bottom
<code>column-reverse</code>	Vertical direction reverse

18. justify-content — सभी important values

Value	क्या करता है
flex-start	Main axis के start पर items
flex-end	Main axis के end पर
center	Main axis center
space-between	Items के बीच equal space; first/last edges पर gap नहीं
space-around	प्रत्येक item के आसपास space
space-evenly	सभी gaps equal
start	Logical start
end	Logical end
left	Left side alignment जहाँ applicable
left	Left side alignment जहाँ applicable
right	Right side alignment जहाँ applicable
normal	Default alignment behavior
safe center	Overflow होने पर safer alignment behavior
unsafe center	Alignment को force करने वाला form

19. align-items — सभी important values

Value	क्या करता है
stretch	Items को cross-axis में stretch करता है, जहाँ applicable
flex-start	Cross-axis start
flex-end	Cross-axis end
center	Cross-axis center
baseline	Text baseline के according align
start	Logical start
end	Logical end
self-start	Item की own writing direction के start के अनुसार
self-end	Item की own writing direction के end के अनुसार
normal	Default behavior



TABLE OF CONTENT

20. align-content — सभी important values

Value	क्या करता है
normal	Default
flex-start	सभी flex lines start पर
flex-end	सभी lines end पर
center	सभी lines center
space-between	Lines के बीच equal space
space-around	Lines के आसपास space
space-evenly	सभी gaps equal
stretch	Lines cross-axis में stretch
start	Logical start
end	Logical end

<code>end</code>	Logical end
<code>baseline</code>	Baseline-based alignment
Important	Explanation
<code>align-content</code> कब visible होता है?	जब multiple flex lines हों, जैसे <code>flex-wrap: wrap</code> , और cross-axis में extra space available हो।
Single-line flex	Single flex line में <code>align-content</code> का सामान्यतः visible effect नहीं होता।

21. `flex-wrap` — सभी values

Value	क्या करता है
<code>nowrap</code>	सभी items को एक line में रखने की कोशिश
<code>wrap</code>	Space खत्म होने पर next line
<code>wrap-reverse</code>	Multiple lines बनती हैं लेकिन cross-axis direction reverse

22. Flex Item Properties

Property	क्या करता है	Example	Deep Explanation	
<code>order</code>	Item का visual order बदलता है	<code>order: 2;</code>	HTML बदले बिना visual arrangement बदल सकते हो।	
<code>flex-grow</code>	Extra space में item कितना grow करे	<code>flex-grow: 2;</code>	Extra available space को grow factors के according distribute करता है।	
<code>flex-shrink</code>	Space कम होने पर item कितना shrink करे	<code>flex-shrink: 2;</code>	Container छोटा होने पर shrinking behavior control करता है।	
<code>flex-basis</code>	Initial main size	<code>flex-basis: 300px;</code>	Flex calculation शुरू करने के लिए initial size देता है।	
<code>flex</code>	Grow + shrink + basis shorthand	<code>flex: 2 2 100px;</code>	तीनों flex sizing properties को एक line में define करता है।	
				
<code>flex</code>	Grow + shrink + basis shorthand	<code>flex: 2 2 100px;</code>	तीनों flex sizing properties को एक line में define करता है।	
<code>align-self</code>	Individual item की cross-axis alignment	<code>align-self: center;</code>	Parent के <code>align-items</code> को individual item के लिए override कर सकता है।	
<code>margin</code>	Item के आसपास space	<code>margin: 10px;</code>	Flex layout में margin भी alignment/available space को प्रभावित कर सकता है।	

23. align-self — सभी important values

Value	क्या करता है
auto	Parent के align-items को follow करता है
normal	Default behavior
stretch	Cross-axis में stretch
flex-start	Cross-axis start
flex-end	Cross-axis end
center	Center
baseline	Baseline alignment
start	Logical start
end	Logical end



self-start

Own writing mode का start

self-end

Own writing mode का end

24. Flex Sizing — Deep Table

Property	Example	Meaning	
<code>flex-basis: 100px</code>	Initial main size = 100px	Flex item की initial size	
<code>flex-grow: 1</code>	Extra space में grow	Extra space मिलने पर item grow करेगा	
<code>flex-grow: 2</code>	Grow factor 2	दूसरे item के <code>grow:1</code> से proportionally ज्यादा extra space ले सकता है	
<code>flex-shrink: 1</code>	Default-like shrink factor	Space कम होने पर shrink कर सकता है	
<code>flex-shrink: 0</code>	No shrinking	Space कम होने पर item को shrink नहीं करने देता	
<code>flex: 1</code>	Flexible item	Common responsive flex pattern	
<code>flex: 0 0 200px</code>	Fixed-ish basis	Grow और shrink दोनों बंद, basis 200px	
<code>flex: 1 1 200px</code>	Flexible 200px basis	Grow + shrink दोनों allowed	
<code>flex: 2 2 100px</code>	Grow 2, shrink 2, basis 100px	तीनों values explicitly set	

25. flex shorthand

Syntax	Equivalent Concept
<code>flex: 2 2 100px</code>	<code>flex-grow:2; flex-shrink:2; flex-basis:100px;</code>
<code>flex: 1 1 auto</code>	Grow + shrink + auto basis
<code>flex: 0 1 auto</code>	No grow, shrink allowed, auto basis
<code>flex: none</code>	Grow/shrink नहीं; fixed basis behavior
<code>flex: auto</code>	Flexible auto-basis behavior
<code>flex: 1</code>	Common flexible-item shorthand

26. Flex gap

Property	Example	क्या करता है
<code>gap</code>	<code>gap: 20px;</code>	Row और column दोनों gaps set करता है।
<code>row-gap</code>	<code>row-gap: 20px;</code>	Flex rows/lines के बीच vertical gap।
<code>column-gap</code>	<code>column-gap: 30px;</code>	Items के बीच horizontal gap।

<code>gap</code> syntax	Meaning	
<code>gap: 20px;</code>	Row + column दोनों 20px	
<code>gap: 10px 20px;</code>	Row gap 10px, column gap 20px	
<code>row-gap: 10px; column-gap: 20px;</code>	Same result explicitly	

27. Flex Alignment Shorthand Methods

Property	क्या combine करता है	Example
<code>flex-flow</code>	<code>flex-direction</code> + <code>flex-wrap</code>	<code>flex-flow: row wrap;</code>
<code>place-content</code>	<code>align-content</code> + <code>justify-content</code>	<code>place-content: center;</code>
<code>place-items</code>	<code>align-items</code> + <code>justify-items</code>	<code>place-items: center;</code>
<code>place-self</code>	<code>align-self</code> + <code>justify-self</code>	<code>place-self: center;</code>

28. Flex में Main Axis और Cross Axis

flex-direction	Main Axis	Cross Axis
row	Horizontal	Vertical
row-reverse	Horizontal	Vertical
column	Vertical	Horizontal
column-reverse	Vertical	Horizontal
Property	किस axis पर काम करती है	
justify-content	Main axis	
align-items	Cross axis	
align-self	Cross axis	
align-content	Multiple flex lines की cross axis distribution	
gap	दोनों directions के gaps	

29. Object/Image Related Properties

Property	क्या काम करता है	Example	Deep Explanation	
<code>object-fit</code>	Image/video को box में fit करता है	<code>object-fit: cover;</code>	<code></code> / <code><video></code> का content अपने box में कैसे fit होगा।	
<code>object-position</code>	Image/video की position	<code>object-position: center;</code>	<code>object-fit: cover</code> के साथ crop area की position control करने में बहुत useful।	
<code>object-view-box</code>	Object content के view region को control करने की modern capability	<code>object-view-box: inset(...);</code>	Advanced replaced-element/SVG presentation use cases में काम आता है।	

30. Visual Effects से जुड़े important properties

Property	क्या करता है	Example	
<code>opacity</code>	Transparency	<code>opacity: .5;</code>	
<code>visibility</code>	Visible/hidden	<code>visibility: hidden;</code>	
<code>filter</code>	Visual filters	<code>filter: blur(5px);</code>	
<code>backdrop-filter</code>	पीछे के content पर filter	<code>backdrop-filter: blur(10px);</code>	
<code>mix-blend-mode</code>	Element को background के साथ blend	<code>mix-blend-mode: multiply;</code>	
<code>isolation</code>	Blending boundary बनाता है	<code>isolation: isolate;</code>	
<code>clip-path</code>	Element का visible shape clip करता है	<code>clip-path: circle(50%);</code>	
<code>mask-image</code>	Mask apply करता है 	<code>mask-image: linear-gradient(...)</code>	

32. Card Hover के लिए Related Properties

Property	Example	Purpose
<code>transform</code>	<code>scale(1.05)</code>	Card को थोड़ा बड़ा
<code>translateY()</code>	<code>translateY(-5px)</code>	Card को ऊपर उठाना
<code>box-shadow</code>	<code>0 10px 30px #0003</code>	Elevation/shadow
<code>transition</code>	<code>all .3s ease</code>	Smooth effect
<code>cursor</code>	<code>pointer</code>	Clickable indication
<code>background-color</code>	<code>#fff</code>	Background change
<code>opacity</code>	<code>.9</code>	Transparency
<code>border-color</code>	<code>blue</code>	Border highlight